Test Plan: RESTful API for Booking Management

1. Introduction

This document outlines the test plan for the RESTful API providing booking management functionality. The API will expose endpoints for creating, reading, updating, and deleting booking records. The focus will be on ensuring functionality, data integrity, performance, and security.

2. Scope

In Scope:

API endpoints for booking creation, retrieval, update, and deletion (CRUD operations).
Data validation for all input fields.
Authentication and authorization mechanisms (although specific implementation details are not defined in this document).
Basic error handling and response codes.
API documentation and usage examples.

Out of Scope:

UI Testing (this plan focuses solely on API testing).
Integration with third-party systems (e.g., payment gateways, mapping services).
Detailed performance testing beyond basic response time checks.

3. Test Environment

Environment	Description	OS	Browser(s)	Hardware Require-ments
QA	Primary testing environment	Windows 10, macOS, Linux	Chrome, Firefox, Edge	Standard Desktop PC
Pre-Prod	Staging environment for final testing	Windows 10, macOS, Linux	Chrome, Firefox, Edge	Similar to QA
Mobile	Testing across a range of mobile devices	Android, iOS	Chrome, Safari	Standard Mobile Phone/ Tablet

devices			

4. Test Strategy & Techniques

We will employ a layered testing approach:

Smoke Testing: Initial tests to verify critical functionality.

Functional Testing: Verification of all API endpoints against defined requirements.

Techniques:

Equivalence Class Partitioning: Dividing input values into groups to minimize test cases.

Boundary Value Analysis: Testing values at the edges of valid ranges.

Decision Table Testing: Systematically covering all combinations of inputs.

Negative Testing: Intentionally invalid inputs to ensure proper error handling.

Usability Testing: Evaluating the ease of use of the API through documentation and example requests.

Exploratory Testing: Leveraging testers' experience to identify unexpected issues and edge cases.

Regression Testing: Re-testing previously verified functionality after code changes.

5. Test Cases (Examples – Detailed Cases will be elaborated)

Endpoint	Operation	Test Case Description	Expected Result	Priority
/bookings	POST	Create a new booking with valid data.	Booking created successfully with unique ID.	High
/bookings/{id}	GET	Retrieve a booking by its ID.	Booking data returned accurately.	High
/bookings/{id}	PUT	Update an existing booking with valid data.	Booking updated successfully, reflected in the system.	High
/bookings/{id}	DELETE	Delete a booking by its	Booking deleted	Medium

		booking by its ID.	deleted successfully, no longer accessible.	
...	...	...	...	...
Boundary Values	...	Test with min/ max values for dates/ amounts etc.	Validation Errors displayed correctly.	High

6. Defect Reporting Procedure

Reporting: All defects will be logged in Jira, including:

- Detailed description
- Steps to reproduce
- Expected vs. actual results
- Environment information (OS, browser, API version)
- Severity and Priority

Triage: The Test Lead will triage defects based on severity (Critical, High, Medium, Low) and priority.

Tracking: Jira will be used to track defect status, assigned developers, and resolution times.

7. Test Schedule & Deliverables

Task	Duration (Sprints)	Resources
Test Plan Creation	1	Test Lead
Test Case Development	2	Testers
Test Case Execution	2	Testers
Summary Report Submission	1	Test Lead

8. Success Criteria

All high-priority test cases pass.

No critical defects remain open at the end of testing.

The API meets performance requirements (defined separately).

API documentation is accurate and complete.

Note: This extended test plan provides a solid foundation. The detailed test cases, specific data validation rules, and performance requirements would be defined in subsequent documentation.